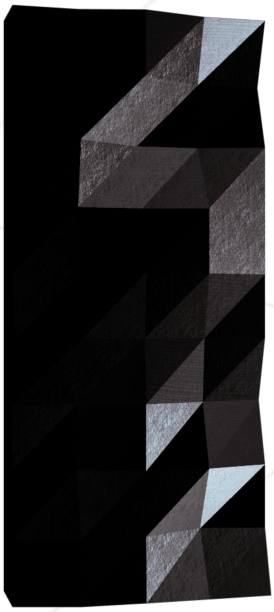

Archivista de Datos

Preservación digital en los Archivos Cibernéticos

Resumen: ¡Únete a los Archivos Cibernéticos como Archivista de Datos! Preserva el conocimiento digital dominando las operaciones con archivos, gestionando flujos de datos y construyendo sistemas de archivo robustos que protejan la información para las generaciones futuras.

Versión: 1.0

Índice general

I. Prólogo **2**

II. Instrucciones sobre la IA **3**

III. Introducción **6**

IV. Recursos de entrenamiento **7**

V. Protocolo de formación de archivistas **8**
- V.1. Principios fundamentales 8
- V.2. Requisitos técnicos 8
- V.3. Librerías autorizadas 9
- V.4. Especificaciones de salida 9

VI. Ejercicio 0: Recuperación de textos antiguos **10**

VII. Ejercicio 1: Creación de archivo **12**

VIII. Ejercicio 2: Gestión de flujos **14**

IX. Ejercicio 3: Seguridad de bóvedas **16**

X. Ejercicio 4: Respuesta a crisis **18**

XI. Entrega y Evaluación **20**

Capítulo I

Prólogo

¡Te damos la bienvenida a los Archivos Cibernéticos, Archivista de Datos!

En el año 2087, el mayor tesoro de la humanidad no es el oro ni los diamantes — son **los datos**. Cada fragmento de conocimiento digital, desde memes antiguos hasta algoritmos cuánticos, se almacena en los vastos Archivos Cibernéticos. Pero los datos son frágiles. Sin una preservación adecuada, se desvanecen para siempre en el vacío digital.

Ahí es donde entras tú. Como Archivista de Datos, eres la persona encargada de proteger a la humanidad frente a la entropía de la información. Tu misión: dominar las artes antiguas de las **operaciones con archivos** para asegurar que ningún conocimiento se pierda jamás.

Los Archivos contienen millones de fragmentos de datos dispersos por distintos sistemas de almacenamiento. Algunos están cifrados en formatos antiguos; otros, corrompidos por el tiempo. Tus herramientas son simples pero poderosas: la capacidad de **abrir** bóvedas de datos, **leer** su contenido, **escribir** nuevos archivos y, lo más importante, manejar lo inesperado sin perder información valiosa.

Toda gran persona archivista empezó donde estás ahora: aprendiendo a comunicarse con los sistemas de almacenamiento, comprendiendo las tres corrientes sagradas del flujo de datos y dominando los rituales de protección que mantienen los datos a salvo. ¡Los Archivos cuentan contigo!

Capítulo II

Instrucciones sobre la IA

Contexto

Durante tu proceso de aprendizaje, la IA puede ayudarte con muchas tareas diferentes. Tómate el tiempo necesario para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyarte con tu trabajo. Sin embargo, siempre debes abordarlas con precaución y evaluar de forma crítica los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca podrás saber con total certeza si tu pregunta está bien formulada o si el contenido generado es el adecuado. Las personas que te rodean son tu recurso más valioso para ayudarte a evitar errores y puntos ciegos.

Mensaje principal:

- Utiliza la IA para reducir las tareas repetitivas o tediosas.
- Desarrolla habilidades de prompting, ya sea para programacion o para otros temas, que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar de forma eficiente y evitar los riesgos comunes, sesgos y problemas éticos.
- Sigue trabajando con tus compañeros para desarrollar tanto habilidades técnicas como habilidades transversales.
- Utiliza únicamente contenido generado por IA que entiendas completamente y del cual puedas responsabilizarte.

Reglas para estudiantes:

- Debes tomarte el tiempo necesario para explorar las herramientas de IA y comprender cómo funcionan, para poder utilizarlas de manera ética y reducir los sesgos potenciales.
- Debes reflexionar sobre tu problema antes de dar instrucciones a la IA. Esto te ayuda a escribir preguntas, instrucciones o conjuntos de datos más claros, detalladas y relevantes utilizando un vocabulario preciso.

- Debes desarrollar el hábito de revisar, cuestionar y probar sistemáticamente cualquier contenido generado por la IA.
- Debes buscar siempre la revisión de otras personas, no te limites a confiar en tu propia validación.

Resultados de esta etapa:

- Desarrollar habilidades de prompting tanto generales como de ámbito especifico.
- Aumentar tu productividad con un uso eficaz de las herramientas de IA.
- Seguir fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

Comentarios y ejemplos:

- Ten en cuenta que la IA puede no tener la respuesta correcta porque esa respuesta no esté ni siquiera en Internet. Además, si te da soluciones incorrectas, intenta no insistir y busca ayuda entre las personas que te rodean. Vas a ahorrarte tiempo y vas a sumar en compresión.
- Vas a enfretarte con frecuencia a situaciones (como exámenes o evaluaciones) donde debes demostrar una comprensión real. Prepárate, sigue construyendo tanto tus habilidades técnicas como transversales.
- Explicar tu razonamiento y debatir con otras personas suele revelar lagunas en tu comprensión de un concepto. Prioriza el aprendizaje entre pares.
- Lo normal es que la herramienta de IA que utilices no conozca tu contexto específico (a menos que se lo indiques), así que te dará respuestas genéricas. Si buscas información más adecuada y más precisa en relación a tu entorno cercano, confía en el resto de estudiantes.
- Donde la IA tiende a generar la respuesta más probable, el resto de estudiantes puede proporcionar perspectivas alternativas y matices valiosos. Confía en la comunidad de 42 como un punto de control de calidad.

✓ Buenas prácticas:

> Le pregunto a la IA: "¿Cómo pruebo una función de ordenación?"Me da algunas ideas. Las pruebo y reviso los resultados con otra persona. Refinamos el enfoque de manera conjunta.

✗ Mala práctica:

> Le pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad. Suspendo mi proyecto.

✓ Buenas prácticas:

> Utilizo la IA para ayudarme a diseñar un parser. Luego, reviso la lógica con otra persona. Encontramos dos errores y lo reescribimos juntos: mejor, más limpio y comprendiendo al 100

✗ Mala práctica:

> Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no puedo justificarlo y suspendo mi proyecto.

Capítulo III

Introducción

¡Te damos la bienvenida al Programa de Entrenamiento de los Archivos Cibernéticos!

La Jefatura de Archivo te ha asignado a la División de Preservación Digital. Tu entrenamiento consta de cinco misiones críticas, cada una desarrollando tu pericia en el manejo de datos:

- **Misión 0**: Recuperación de textos antiguos - Recupera datos de unidades de almacenamiento antiguas.
- **Misión 1**: Creación de archivo - Establece nuevos protocolos de preservación de datos.
- **Misión 2**: Gestión de flujos - Domina los tres canales de datos.
- **Misión 3**: Seguridad de bóvedas - Implementa procedimientos de almacenamiento a prueba de fallos.
- **Misión 4**: Respuesta a crisis - Gestiona archivos corrompidos y fallos de acceso.

Cada misión simula escenarios reales de archivo. Los sistemas con los que trabajarás pueden parecer simples, pero forman la base de toda preservación digital. Domina estos fundamentos y podrás proteger la herencia digital de la humanidad.

`PROTOCOLO DE ARCHIVO: Todas las operaciones deben seguir estándares estrictos de preservación de datos. Archivos corrompidos significan conocimiento perdido para siempre.`

Capítulo IV

Recursos de entrenamiento

Los Archivos proporcionan materiales esenciales de entrenamiento en el directorio `attachments/`:

- **Archivo de herramientas**: Extrae `data-generator-tools.tar.gz` para acceder al conjunto de herramientas de entrenamiento.
- **Generador de datos**: Ejecuta `python3 data_generator.py` para crear los archivos de entrenamiento necesarios para el **Ejericio 0** (ancient_fragment.txt) y el **Ejercicio 4** (standard_archive.txt y otros archivos).
- **Datos de ejemplo**: Consulta los ejemplos disponibles en `data_samples/` para comprender los formatos y estructuras de datos esperados.
- **Configuración JSON**: El archivo `sample_data.json` contiene metadatos e información de escenarios de prueba solo con fines de referencia.

Extrae primero el archivo de herramientas: tar -xzf data-generator-tools.tar.gz.
Después genera tus datos de entrenamiento antes de comenzar los ejercicios. Los protocolos de seguridad de los Archivos requieren estructuras de archivos y formatos de contenido específicos para una evaluación adecuada.

Capítulo V

Protocolo de formación de archivistas

Te damos la bienvenida al Programa de Entrenamiento de los Archivos Cibernéticos. Como postulante a Archivista de Datos, dominarás las artes antiguas de la manipulación de archivos y la gestión de recursos que han protegido el conocimiento digital durante siglos.

V.1. Principios fundamentales

El Código de la persona Archivista: cada operación con archivos debe ser segura, cada recurso correctamente gestionado y cada error manejado con elegancia. Los Archivos han sobrevivido a catástrofes digitales porque las personas archivistas siguen estos protocolos sagrados.

Los ejemplos proporcionados en cada ejercicio muestran el comportamiento esperado. Estúdialos con atención: contienen la sabiduría de quienes han ejercido el arte del archivo antes que tú.

V.2. Requisitos técnicos

Tus programas de entrenamiento deben demostrar dominio de:

- **Operaciones con archivos**: lectura y escritura en bóvedas de almacenamiento.
- **Gestión de flujos**: uso adecuado de los canales de entrada, salida y alerta.
- **Gestores de contexto**: el sagrado protocolo `with` para la seguridad de recursos.
- **Manejo de errores**: respuesta a crisis utilizando protocolos `try-except`.

V.3. Librerías autorizadas

Para las operaciones de procesamiento de datos, se autoriza el uso de los recursos siguientes:

- **Librería estándar**: módulo `sys` para la gestión de flujos.
- **Procesamiento JSON**: módulo `json` para manejo de datos estructurados.
- **Funciones integradas**: `open()`, `read()`, `write()`, `close()`, `print()`, `input()`.

No cerrar correctamente los manejadores de archivos ha causado corrupción de datos a lo largo de la historia. Usa siempre la sentencia with al acceder a las bóvedas de almacenamiento - no es solo una buena práctica, es cuestión de supervivencia.

V.4. Especificaciones de salida

Cada ejercicio proporciona ejemplos de terminal que muestran el formato de salida esperado. Aunque la estructura básica debe mantenerse, puedes personalizar los mensajes para reflejar tu comprensión de las operaciones de los Archivos, siempre que la información esencial se conserve.

Consejo de Archivista Pro: es tarea de archivista especialista imprimir su personalidad en los mensajes del sistema manteniendo estándares profesionales. ¡Haz que los Archivos se sientan vivos con tu toque único!

¿Qué hace buena a una persona Archivista de Datos? Quien comprende que cada archivo es una pieza de la historia digna de preservación, y que cada operación es una responsabilidad con las generaciones futuras.

Capítulo VI

Ejercicio 0: Recuperación de textos antiguos

	Ejercicio: 0
ft_ancient_text	
Directorio de entrega: *ex0/*	
Archivos a entregar: `ft_ancient_text.py`	
Funciones autorizadas: `open()`, `read()`, `close()`, `print()`	

Encargo de la misión: los Archivos han descubierto un fragmento de datos antiguos en la Bóveda 7. Tu primera tarea como Archivista de Datos es recuperar esta valiosa información antes de que se degrade más. La leyenda dice que contiene sabiduría de la era en la que quien desarrollaba ¡realmente escribía documentación!

Fuente de datos: usa el generador de datos proporcionado para crear tu archivo de prueba:

Ejecuta:
python3 tools/data_generator.py para crear ancient_fragment.txt con los datos requeridos.
Tu programa debe leer exactamente este nombre de archivo; los protocolos de seguridad de los Archivos son muy específicos con las convenciones de nombres.

Protocolo de recuperación: accede a la unidad de almacenamiento que contiene `ancient_fragment.txt`, extrae todos los datos preservados y muestra el proceso de recuperación. Piénsalo como arqueología digital — estás desenterrando sabiduría perdida desde las profundidades de la matriz de almacenamiento.

Si la bóveda es inaccesible, tu programa debe mostrar: ERROR: Storage vault not found. Run data generator first. Recuerda: quien se precie de ser archivista siempre comprueba si la bóveda existe antes de intentar el acceso. Intentar leer archivos inexistentes es como intentar abrir una puerta que no está ahí - nunca termina bien.

Salida esperada: tu programa debe mostrar la cabecera del sistema, el estado de acceso a la bóveda, los datos recuperados con el formato adecuado y la confirmación de finalización. El ejemplo de terminal muestra el formato exacto, pero puedes añadir tu propia personalidad de archivista a los mensajes siempre que la información esencial se conserve.

Registro de recuperación:

```
$> python3 ft_ancient_text.py
=== CYBER ARCHIVES - DATA RECOVERY SYSTEM ===

Accessing Storage Vault: ancient\_fragment.txt
Connection established...

RECOVERED DATA:
{[}FRAGMENT 001{]} Digital preservation protocols established 2087
{[}FRAGMENT 002{]} Knowledge must survive the entropy wars
{[}FRAGMENT 003{]} Every byte saved is a victory against oblivion

Data recovery complete. Storage unit disconnected.
```

¿Qué le sucede al sistema de almacenamiento si las conexiones no se cierran correctamente? ¿Por qué es crítico el protocolo de desconexión?

Capítulo VII

Ejercicio 1: Creación de archivo

s

	Ejercicio: 1
ft_archive_creation	
Directorio de entrega: $ex1/$	
Archivos a entregar: `ft_archive_creation.py`	
Funciones autorizadas: `open()`, `write()`, `close()`, `print()`	

Encargo de la misión: ¡Excelente trabajo con la recuperación de datos! La Jefatura de Archivo está impresionada. Tu siguiente tarea: establecer un nuevo protocolo de preservación creando entradas de archivo nuevas. Es hora de hacer historia en lugar de solo leerla.

Protocolo de creación de archivo: establece una nueva unidad de almacenamiento llamada `new_discovery.txt` e inscribe tres entradas de datos críticas: información sobre un avance en algoritmos cuánticos, métricas de mejora de rendimiento (347 % de ganancia de eficiencia) e identificación de la persona archivista. Piénsalo como escribir una cápsula del tiempo digital para las generaciones futuras.

Recuerda que el modo de preservación crea archivos nuevos o reemplaza los existentes. A diferencia de la lectura, escribir es permanente - una vez que sellas la bóveda, los datos pasan a formar parte de los Archivos eternos.

Salida esperada: tu programa debe mostrar la cabecera del sistema, la inicialización de la unidad de almacenamiento, la inscripción de datos con entradas numeradas y la confirmación de finalización. El ejemplo de terminal muestra el formato esperado, pero siéntete libre de añadir tus propios adornos archivísticos.

¡Ten cuidado con las operaciones de escritura! En los Archivos reales, sobrescribir accidentalmente datos históricos se considera un crimen contra el propio conocimiento. Verifica siempre los nombres de tus archivos.

Registro de archivo:

```
$> python3 ft_archive_creation.py
=== CYBER ARCHIVES - PRESERVATION SYSTEM ===

Initializing new storage unit: new_discovery.txt
Storage unit created successfully...

Inscribing preservation data...
{[}ENTRY 001{]} New quantum algorithm discovered
{[}ENTRY 002{]} Efficiency increased by 347%
{[}ENTRY 003{]} Archived by Data Archivist trainee

Data inscription complete. Storage unit sealed.
Archive 'new_discovery.txt' ready for long-term preservation.
```

¿Cuál es la diferencia crítica entre el modo de extracción ('r') y el modo de preservación ('w')? ¿Por qué es vital esta distinción para las personas archivistas?

Capítulo VIII

Ejercicio 2: Gestión de flujos

	Ejercicio: 2
ft_stream_management	
Directorio de entrega: *ex2/*	
Archivos a entregar: `ft_stream_management.py`	
Funciones autorizadas: `sys.stdin, sys.stdout, sys.stderr, input(), print(), import sys`	

Encargo de la misión: ¡Trabajo de archivo sobresaliente! La Jefatura de Archivo tiene un nuevo reto para ti. Los Archivos operan mediante tres canales sagrados de datos activos desde la fundación de la civilización digital. Estos canales son más antiguos que Internet: el equivalente digital de las rutas comerciales antiguas.

Protocolo de comunicación: accede a los tres canales sagrados de datos de los Archivos — flujo de entrada, canal estándar y canal de alertas. Recopila identificación de la persona archivista e información de estado y demuestra la separación correcta de canales dirigiendo distintos tipos de mensajes a sus flujos correspondientes.

Piensa en los flujos como frecuencias de comunicación diferentes: stdin para recibir mensajes, stdout para emisiones normales y stderr para alertas de emergencia. Cada uno tiene su propósito en la gran red de comunicación.

Salida esperada: tu programa debe mostrar la cabecera del sistema, recopilar entrada de la persona usuaria para el ID de archivista y el informe de estado, y después producir mensajes usando los flujos correctos: mensajes estándar vía `sys.stdout` y alertas vía `sys.stderr`. Completa con una confirmación de prueba de comunicación exitosa.

¡Nunca mezcles tus flujos! Enviar alertas por el canal estándar es como susurrar "¡FUEGO!" en una biblioteca - anula el propósito y confunde a todo el mundo.

Registro de comunicación:

```
$> python3 ft_stream_management.py
=== CYBER ARCHIVES - COMMUNICATION SYSTEM ===

Input Stream active. Enter archivist ID: ARCH_7742
Input Stream active. Enter status report: All systems nominal

{[}STANDARD{]} Archive status from ARCH_7742: All systems nominal
{[}ALERT{]} System diagnostic: Communication channels verified
{[}STANDARD{]} Data transmission complete

Three-channel communication test successful.
```

¿Por qué los Archivos mantienen canales separados para datos estándar y alertas? ¿Qué podría ocurrir si se mezclaran estos flujos?

Capítulo IX

Ejercicio 3: Seguridad de bóvedas

	Ejercicio: 3
ft_vault_security	
Directorio de entrega: *ex3/*	
Archivos a entregar: `ft_vault_security.py`	
Funciones autorizadas: `open()`, `read()`, `write()`, `with statement`, `print()`	

Encargo de la misión: ¡Impresionantes habilidades de comunicación! La Jefatura de Archivo ha notado tu potencial y te asciende a operaciones de Seguridad de Bóvedas. Aquí es donde las verdaderas personas archivistas se ponen a prueba. Comienza la etapa mayor, donde un solo error puede corromper siglos de conocimiento.

Los Archivos han aprendido de siglos de pérdida de datos. El antiguo protocolo **with** se desarrolló tras la Gran Corrupción de Datos de 2089, cuando miles de archivos se perdieron porque las conexiones no se cerraban correctamente durante fallos del sistema.

El Protocolo Sagrado: cuando abres una bóveda con la sentencia **with**, suceden tres cosas automáticamente: la bóveda se abre, tus operaciones se ejecutan de forma segura dentro de su barrera protectora y la bóveda se sella, aunque algo salga mal. Es como tener escolta digital para tus archivos.

Protocolo de seguridad de bóvedas: implementa que las operaciones de archivos sean seguras usando la sentencia **with** tanto para leer datos clasificados como para preservar información nueva. Tu programa debe demostrar el sellado automático de la bóveda independientemente del éxito o fallo de la operación, con un registro de seguridad profesional en todo momento.

Esta es la esencia de la gestión segura de recursos: adquirir, usar y liberar de forma garantizada. Sin la sentencia with, básicamente dejas puertas de bóveda abiertas en un huracán digital. No seas esa clase de archivista.

Registro de seguridad:

```
$> python3 ft_vaul_security.py
=== CYBER ARCHIVES - VAULT SECURITY SYSTEM ===

Initiating secure vault access...
Vault connection established with failsafe protocols

SECURE EXTRACTION:
{[}CLASSIFIED{]} Quantum encryption keys recovered
{[}CLASSIFIED{]} Archive integrity: 100%

SECURE PRESERVATION:
{[}CLASSIFIED{]} New security protocols archived
Vault automatically sealed upon completion

All vault operations completed with maximum security.
```

¿Cómo previene el protocolo **with** la corrupción de datos? ¿Qué es el principio RAII y por qué es crucial para la seguridad de las bóvedas?

Capítulo X

Ejercicio 4: Respuesta a crisis

	Ejercicio: 4
ft_crisis_response	
Directorio de entrega: *ex4/*	
Archivos a entregar: `ft_crisis_response.py`	
Funciones autorizadas: `open()`, `read()`, `write()`, `with statement`, `try/except`, `print()`	

Encargo de la misión: ¡Trabajo excepcional en seguridad de bóvedas! Has obtenido el nivel más alto de autorización. La Jefatura de Archivo tiene una última prueba: operaciones de Respuesta a Crisis. Esta es la prueba definitiva, donde las personas archivistas demuestran que pueden manejar el caos de los desastres de datos del mundo real.

Preparación de datos:

Ejecuta: `python3 tools/data_generator.py` para crear archivos de prueba, incluidos `standard_archive.txt` y otros archivos de prueba. Tu programa probará el acceso a varios archivos para simular diferentes escenarios de crisis.

Protocolo de respuesta a crisis: desarrolla un sistema integral de gestión de crisis que implemente una función de manejo de crisis para operaciones de archivo. El sistema debe gestionar de forma adecuada distintos tipos de fallos de acceso a archivos utilizando protocolos a prueba de fallos combinados con la sentencia **with** para evitar la corrupción de datos durante los errores.

En los Archivos reales, las crisis ocurren a diario: archivos corrompidos, datos perdidos, brechas de seguridad. Quien se considere archivista especialista no entra en pánico, sino que responde con precisión y calma, asegurando que el sistema permanezca estable sin importar el caos que se desate.

Categorías de crisis: gestiona los archivos perdidos en la matriz de almacenamiento, violaciones de protocolos de seguridad, anomalías inesperadas del sistema y operaciones exitosas. Prueba con intentos de acceso a archivos inexistentes, bóvedas restringidas por seguridad y operaciones estándar de recuperación de archivos.

Salida esperada: tu programa debe mostrar alertas de crisis para cada intento de acceso, mensajes de respuesta adecuados según los tipos de error (`FileNotFoundError`, `PermissionError` u otras excepciones), confirmaciones de estado y finalización global de seguridad. El ejemplo de terminal muestra el formato esperado—síguelo con precisión para mantener la consistencia en la red de los Archivos.

Registro de crisis:

```
$> python3 ft_crisis\response.py
=== CYBER ARCHIVES - CRISIS RESPONSE SYSTEM ===

CRISIS ALERT: Attempting access to 'lost_archive.txt'...
RESPONSE: Archive not found in storage matrix
STATUS: Crisis handled, system stable

CRISIS ALERT: Attempting access to 'classified_vault.txt'...
RESPONSE: Security protocols deny access
STATUS: Crisis handled, security maintained

ROUTINE ACCESS: Attempting access to 'standard_archive.txt'...
SUCCESS: Archive recovered - ``Knowledge preserved for humanity''
STATUS: Normal operations resumed

All crisis scenarios handled successfully. Archives secure.
```

¿Cuáles son las amenazas más peligrosas para los archivos digitales? ¿Cómo previene la respuesta adecuada a crisis la pérdida de datos y mantiene la estabilidad del sistema?

Capítulo XI

Entrega y Evaluación

Entrega tu trabajo en tu repositorio `Git` como de costumbre. Solo se evaluará el contenido dentro de tu repositorio durante la defensa. No dudes en verificar que los nombres de tus archivos sean los correctos.

Durante la evaluación, pueden pedirte que expliques operaciones con archivos, que demuestres el manejo de errores o que muestres cómo funciona la sentencia with. Asegúrate de comprender los conceptos detrás de cada ejercicio.

Debes entregar únicamente los archivos solicitados por el subject de este proyecto. Concéntrate en escribir código limpio y simple que demuestre claramente tu comprensión de las operaciones con archivos.